

Fault-Tolerant Multimodal Presence Verification and Safe Activation Policies on ESP32-S3 Using mmWave Radar and Vision

Shobhit Singh

Department of Electrical and Computer Engineering
Iowa State University

May 11, 2026

Project Report
CPRE 5450: Dependable Computing Systems
Iowa State University
Spring 2026

ABSTRACT

Assistive room-scale embedded systems increasingly rely on edge perception to decide when to activate high-power sensing and interaction (camera inference, audio capture, actuation). However, these systems face frequent faults and uncertainties including lighting-induced vision degradation, radar clutter, transient communication errors, timing variability, and resource constraints. A single-modality activation scheme is therefore brittle and can produce two critical failure modes: (i) false activations that waste energy and may raise privacy concerns, and (ii) missed activations that deny assistance and reduce system trustworthiness.

This project proposes a fault-tolerant activation subsystem implemented on an ESP32-S3 platform integrating a camera module and a 24 GHz FMCW mmWave radar module. The approach uses heterogeneous redundancy and system-level diagnosis: mmWave provides presence cues robust to lighting and can be privacy-preserving, while vision provides spatial confirmation and helps resolve radar ambiguities. The subsystem performs cross-sensor consistency checks and temporal redundancy to detect inconsistencies, classifies them into transient versus persistent faults, and selects recovery actions accordingly.

A two-tier safe-state policy is introduced. In a *soft safe state*, the system suppresses full activation and performs re-validation under a bounded time window; in a *hard safe state* (persistent inconsistency), the system blocks activation, isolates low-confidence inputs, and optionally provides user feedback while maintaining minimal power consumption. Design choices are guided by multi-source sensor fusion research that explicitly addresses cross-sensor inconsistency and uncertainty quantification, and by radar presence-detection research that motivates OOD-aware monitoring for cluttered indoor environments.

Evaluation will combine controlled fault injection (sensor packet drops, radar framing errors, camera frame corruption/blur, induced timing delays) with environmental stress tests (low light, reflective surfaces, moving clutter). Metrics include false activation rate, missed activation rate, time-to-detect inconsistency, fault coverage, and activation latency/energy overhead relative to a non-fault-tolerant baseline.

This work demonstrates how classical fault-tolerance principles—redundancy, error detection, diagnosis, and graceful degradation—can be concretely applied to practical edge AI activation pipelines on resource-constrained embedded hardware, improving reliability and trustworthiness for assistive room settings.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Scope of This Report	4
1.3	The Activation Boundary as a Safety Boundary	4
1.4	Report Organization	5
2	Problem Statement and Failure Modes	5
2.1	The Brittleness of Single-Modality Activation	5
2.2	System-Level Failure Modes	5
3	Hardware Platform	6
3.1	ESP32-S3 Edge Device	6
3.2	24 GHz mmWave Radar Sensor	7
3.3	Camera Module	7
3.4	Design Constraint: Resource-Constrained Embedded System	7
4	Known Approaches and Fault-Tolerance Analysis	7
4.1	Approach 1: mmWave-Only Presence Sensing	7
4.2	Approach 2: Camera-Only or Vision-Heavy Activation	8
4.3	Approach 3: Naive Camera + mmWave Fusion	8
4.4	Approach 4: Uncertainty-Aware Sensor Fusion	9
4.5	Approach 5: OOD Detection and Fault Injection	9
5	Proposed Fault-Tolerant Architecture	9
5.1	Design Philosophy	9
5.2	Architecture Overview	10
5.3	Architecture Blocks	10
5.3.1	Radar Input Path	10
5.3.2	Camera Input Path	11
5.3.3	Fault Diagnosis Layer	11
5.3.4	Fault Classifier	11
5.4	ESP32-S3 FreeRTOS Task Model	12
6	Fault Model and Diagnosis Logic	12
6.1	Fault Taxonomy	12
6.1.1	By Source	12
6.1.2	By Duration	12
6.1.3	SDC-Like (Silent Data Corruption) Faults	13
6.2	Fault Detection Rules	13
6.3	Diagnosis Decision Flow	13

7	Safe-State and Recovery Policy	14
7.1	State Machine Overview	14
7.2	Soft Safe State	15
7.3	Hard Safe State	15
7.4	Recovery Block for Visual Confirmation	16
7.5	Confidence Score Fusion	16
8	Evaluation Plan and Metrics	16
8.1	Evaluation Design	16
8.2	Fault Injection Tests	17
8.3	Environmental Stress Tests	17
8.4	Metrics	18
8.5	Expected Results	18
9	Analysis of Existing Solutions vs. Proposed Improvement	18
10	Expected Contribution and Limitations	19
10.1	Expected Contributions	19
10.2	Why This Matters	20
10.3	Limitations	20
11	Conclusion	20

1 Introduction

1.1 Motivation

Room-scale assistive systems must decide when to wake up, when to sense, and when to interact. This decision is especially critical in privacy-preserving systems for blind or visually impaired (BVI) users: unnecessary camera or audio activation can expose private data, while missed activation can deny assistance at a moment of need.

The broader context for this work is a multimodal assistive perception platform (CPRE 5750/HCI 5750) running on a Saeed XIAO ESP32-S3 Sense. That system integrates a camera, microphone, 24 GHz mmWave presence sensor, pan-tilt motors, and an Android companion for gesture/audio interaction and user tracking. Its design philosophy is already privacy-centered: local edge perception, gesture-controlled recording, minimal data transmission, and camera/audio activation only when the user intentionally engages.

1.2 Scope of This Report

This CPRE 5450 report isolates one dependability-critical part of the larger system: the **activation and presence-verification subsystem**. The question studied is:

When is the system allowed to activate high-power or privacy-sensitive functions such as camera inference, audio capture, or servo actuation?

This is not only a sensor-fusion problem—it is a **fault-tolerant decision problem**. The subsystem must remain reliable under sensor faults, timing faults, and environmental uncertainty. The report therefore focuses on fault detection, diagnosis, and safe activation rather than full object recognition or full assistant behavior.

1.3 The Activation Boundary as a Safety Boundary

Activation is treated as the safety boundary of the system:

- A **false positive** (false activation) means the system activates camera, audio, or actuation when no valid user is present. This wastes battery and processing resources and may create privacy concerns.
- A **false negative** (missed activation) means the system does not activate when a user actually needs assistance. This is a denial-of-service-style failure for an assistive system.

The CPRE 5450 work therefore focuses on making this activation decision dependable under sensor faults, timing faults, and environmental uncertainty, using the fault-tolerance principles taught in the course: redundancy, error detection, diagnosis, graceful degradation, and safe states.

1.4 Report Organization

Section 2 defines the problem and failure modes. Section 3 describes the hardware platform. Section 4 surveys known approaches. Section 5 presents the proposed fault-tolerant architecture. Section 6 details the fault model and diagnosis logic. Section 7 describes the safe-state and recovery policy. Section 8 defines the evaluation plan and metrics. Section 9 compares existing solutions to the proposed improvement. Section 10 states expected contributions and limitations. Section 11 concludes.

2 Problem Statement and Failure Modes

2.1 The Brittleness of Single-Modality Activation

A single-modality activation scheme is brittle because each sensor has fundamentally different failure conditions:

- A **camera** may fail under low light, motion blur, partial occlusion, or frame-buffer problems. The official ESP32 camera driver documentation warns that non-JPEG formats can stress PSRAM and that image data may be lost, especially under heavier memory/bus load [9].
- A **24 GHz mmWave radar** provides lighting-independent presence detection, but it can still be affected by clutter, reflections from walls or furniture, moving curtains, fans, or incorrect range-gate tuning. The Seeed sensor documentation explicitly identifies installation environment and moving non-human objects as factors that can affect detection behavior [10].

2.2 System-Level Failure Modes

Using the classical fault-error-failure chain [1], the system-level failures of concern are:

False Activation. The system activates camera, audio, or actuation even though no valid user is present. This wastes battery and processing resources and may create privacy concerns.

Missed Activation. The system fails to activate even though the user is present and needs interaction. This is a denial-of-service-style failure for an assistive system.

Table 1 maps concrete fault sources through error states to system-level failures.

Table 1: Fault–error–failure chain for the activation subsystem.

Fault Source	Error State	System Failure	Consequence
Low light / occlusion	Camera false negative	Missed activation	No assistance to user
Fan / curtain / reflection	Radar false positive	False activation	Battery drain + privacy risk
UART packet drop	Stale radar state	Wrong decision	Unsafe or unavailable behavior
ESP32 task delay	Old camera frame used	Silent wrong decision	Hard-to-detect reliability failure

3 Hardware Platform

3.1 ESP32-S3 Edge Device

The hardware basis is a **Seeed XIAO ESP32-S3 Sense**, a compact (21×17.5 mm) module based on the Espressif ESP32-S3 SoC. The ESP32-S3 is not a generic microcontroller; it provides a set of features relevant to both perception and fault tolerance:

Table 2: ESP32-S3 hardware specification.

Component	Specification
CPU	Xtensa LX7 dual-core, up to 240 MHz
SRAM	512 KB on-chip
PSRAM	8 MB Octal SPI (MALLOC_CAP_SPIRAM)
Flash	8 MB
Peripherals	UART, I2S, I2C, SPI, PWM (LEDC)
Security	Secure boot v2, flash encryption, digital signature, HMAC, World Controller
Wireless	802.11b/g/n 2.4 GHz Wi-Fi; Bluetooth 5.0 LE
Camera I/F	DVP parallel (8-bit + VSYNC + HREF + PCLK)
SDK	ESP-IDF v5/v6 (FreeRTOS)

The vector instructions and dual-core design support lightweight signal processing without a dedicated hardware accelerator. The security subsystem (secure boot, flash encryption) is relevant for trustworthy operation: a compromised firmware that silently disables fault-detection logic would undermine all dependability guarantees. The ESP-IDF **Task Watchdog Timer (TWDT)** can monitor FreeRTOS tasks, detect stalls, and trigger recovery—making it directly useful for timing-fault detection [8].

However, the ESP32-S3 is resource-constrained compared with a laptop or smartphone. This motivates a **lightweight activation controller** instead of a heavy deep-learning pipeline.

3.2 24 GHz mmWave Radar Sensor

The presence sensor is the **Seeed 24 GHz mmWave Human Static Presence Module** [10]. Key characteristics:

Table 3: 24 GHz mmWave sensor specifications.

Parameter	Value
Sensing principle	FMCW (Frequency-Modulated Continuous Wave)
Detection range	0.75–6 m (adjustable via range gate)
Detection angle	$\pm 60^\circ$
Distance resolution	0.75 m
Interface	UART (3.3 V logic)
Detects	Moving and stationary human presence
Power	Low-power sentinel mode available

The mmWave sensor is attractive as a low-power primary sentinel because it can detect human presence independent of lighting conditions, does not directly capture identifiable images (privacy-preserving), and consumes significantly less power than running the camera continuously. Its limitations—sensitivity to clutter and environmental objects—motivate using it as a trigger rather than a sole decision-maker.

3.3 Camera Module

The ESP32-S3 Sense includes an OmniVision OV2640 camera (2 MP, DVP interface). In the activation context, the camera runs briefly for visual confirmation after the radar triggers. Frame resolution is constrained to 240×240 RGB565 for on-chip processing (approximately 15 fps raw, 5–10 fps effective with inference).

3.4 Design Constraint: Resource-Constrained Embedded System

The ESP32-S3 has no hardware GPU or neural-network accelerator. All inference runs on the LX7 cores using INT8-quantized models via Espressif’s ESP-DL library. Per-frame inference takes 100–200 ms, which interacts with sensor freshness requirements in the fault-detection logic. Memory pressure (512 KB SRAM, 8 MB PSRAM) means the fault-detection logic must be lightweight—threshold-based and rule-based rather than deep learning.

4 Known Approaches and Fault-Tolerance Analysis

4.1 Approach 1: mmWave-Only Presence Sensing

mmWave radar is attractive for indoor presence detection because it is contactless, lighting-independent, and more privacy-preserving than always-on vision. Li and Hishiyama [2] studied indoor people detection and tracking using mmWave radar and sub-sensors, showing that mmWave

is feasible for indoor monitoring but has limitations under multi-person and real-room conditions. Zhang et al. [6] further demonstrated a robust mmWave framework for people counting and motion classification.

Strengths: mmWave sensing works without visible light, can detect motion and micro-motion, and does not directly capture identifiable images.

Weaknesses: Radar-only activation is vulnerable to clutter, reflections, static/moving ambiguity, and nuisance motion from fans or curtains. The Sreed sensor documentation explicitly identifies moving non-human objects and strong reflectors as environmental factors [10].

Fault-tolerance analysis: Radar-only activation is efficient, but it has weak *diagnosability*. If radar reports “presence,” the system has no independent modality to determine whether the cause is a true person, clutter, or sensor fault. Single-modality systems cannot perform cross-sensor consistency checks.

4.2 Approach 2: Camera-Only or Vision-Heavy Activation

Camera-based sensing provides semantic information that radar cannot easily provide: face/body detection, gesture recognition, object localization, and user tracking.

Strengths: Vision can confirm spatial structure, body/face cues, and user intent. It supports higher-level interaction such as open-palm activation, pointing gestures, and visual confirmation.

Weaknesses: Camera-only activation is sensitive to lighting, occlusion, motion blur, privacy concerns, and memory/processing limits. On the ESP32-S3, camera processing competes with other real-time tasks. The official ESP32 camera driver documentation warns that non-JPEG formats can stress PSRAM and that image data may be lost under heavier bus load [9].

Fault-tolerance analysis: Vision-heavy activation gives richer information, but it is expensive and fragile on a constrained MCU. It should be used as a *verifier*, not as an always-on primary trigger.

4.3 Approach 3: Naive Camera + mmWave Fusion

Prior research supports combining mmWave and cameras for indoor occupancy sensing. Li and Hishiyama [2] showed that camera assistance improved detection accuracy beyond a radar-only scheme. Radar and camera provide complementary information: radar handles darkness better; vision handles semantic ambiguity better.

Strengths: Fusion improves robustness because camera and radar fail differently (heterogeneous redundancy).

Weaknesses: Naive fusion is not automatically fault-tolerant. Simple AND logic may increase missed activations if one sensor temporarily fails. Simple OR logic may increase false activations if one sensor is noisy. Neither rule diagnoses *why* the sensors disagree.

Fault-tolerance analysis: The project needs **health-aware fusion**, not just sensor fusion. The decision layer must track confidence, freshness, disagreement, and fault persistence.

4.4 Approach 4: Uncertainty-Aware Sensor Fusion

Yu et al. [3] propose a multi-source sensor correlation adaptive fusion framework (MSCAF) with uncertainty quantification for intelligent fault diagnosis. Their work argues that multi-source sensor fusion is promising, but reliability is challenged by variations in signal quality and inconsistencies across sensors. The MSCAF framework uses Dempster–Shafer evidence theory and Dirichlet-distribution-based uncertainty modeling to split sensors into high-confidence and low-confidence groups and weight them adaptively.

Strengths: This approach directly supports the idea that sensors should not always be trusted equally. A low-confidence or inconsistent sensor should be down-weighted rather than blindly fused.

Weaknesses: The full MSCAF method is likely too heavy for an ESP32-S3 implementation because it uses deep feature extraction and graph-based modeling designed for richer industrial datasets.

How it supports this project: This project adapts the concept, not the full algorithm. Instead of implementing full DS/Dirichlet fusion, the design uses a lightweight confidence score:

$$C_i = w_s S_i + w_f F_i + w_t T_i \quad (1)$$

where S_i is sensor agreement score, F_i is freshness/timestamp validity, and T_i is task/transport health. This is computable in microseconds on the ESP32-S3.

4.5 Approach 5: OOD Detection and Fault Injection

Kahya et al. [5] (HOOD) study real-time human presence and out-of-distribution detection using FMCW radar. HOOD frames the problem as not only detecting humans but also rejecting out-of-distribution radar inputs such as clutter or abnormal environmental conditions. This directly motivates OOD-aware thresholding in the radar health monitor.

Moradi and Denil [4] describe machine-learning-assisted model-implemented fault injection for validation. Their work supports the idea that validation should deliberately vary the type, location, and timing of injected faults—which is how this project’s evaluation is designed.

Fault-tolerance analysis: OOD detection and fault injection move the project from “works in normal conditions” to “tested under abnormal conditions,” which is the appropriate standard for a dependable system.

5 Proposed Fault-Tolerant Architecture

5.1 Design Philosophy

The proposed architecture treats activation as a **fault-tolerant state machine**. The mmWave sensor acts as a low-power sentinel. When radar indicates possible presence, the camera is activated briefly for confirmation. A fusion layer evaluates agreement, confidence, freshness, and timing health before allowing full activation.

The guiding principle is: *when uncertain, fail into non-activation, not unsafe activation*. This is a **fail-safe** design: ambiguity triggers a safe state, not an active state.

5.2 Architecture Overview

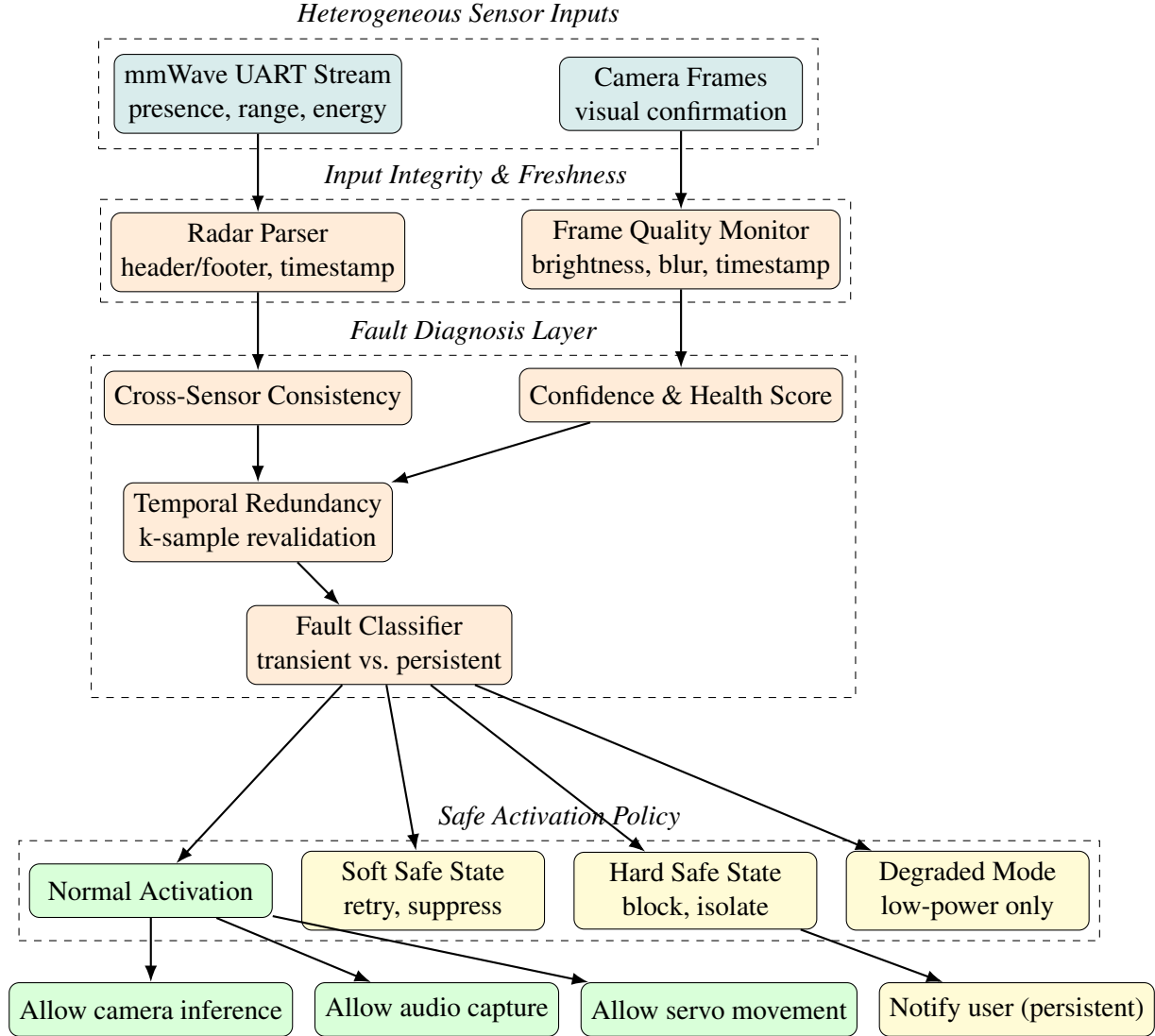


Figure 1: Proposed fault-tolerant activation subsystem architecture.

5.3 Architecture Blocks

5.3.1 Radar Input Path

The mmWave sensor communicates over UART at 3.3 V logic. The **Radar Parser** processes each incoming frame:

- Validates frame header and footer bytes (integrity check).
- Timestamps the frame on arrival (`esp_timer_get_time()`).
- Extracts range-gate energy and presence state.

- Marks the frame stale if it arrives after a configurable deadline.

A packet-drop counter tracks consecutive missing frames. After N_{drop} consecutive drops, the parser flags a UART transport fault.

5.3.2 Camera Input Path

The **Frame Quality Monitor** validates each captured frame:

- **Timestamp check:** frame age $> T_{\text{stale}}$ (e.g., 250 ms) flags a timing fault.
- **Brightness gate:** mean pixel luminance below L_{min} flags a low-light degradation.
- **Blur estimate:** variance of a Laplacian edge filter below σ_{min}^2 flags motion blur.

Frame quality scores feed into the per-sensor confidence score C_i (Equation 1).

5.3.3 Fault Diagnosis Layer

The diagnosis layer performs three operations in sequence each decision window:

1. **Cross-sensor consistency check.** If radar reports presence and camera reports no user (or vice versa), an inconsistency flag is raised. The diagnosis layer does not immediately decide which sensor is correct.
2. **Confidence and health scoring.** Each sensor is assigned a score $C_i \in [0, 1]$ using Equation 1. Weights w_s, w_f, w_t are tunable; a default configuration is $w_s = 0.5, w_f = 0.3, w_t = 0.2$.
3. **Temporal redundancy.** A sliding window of k consecutive decision samples is maintained. A fault is classified as *transient* if inconsistency occurred in fewer than k_{hard} of the last k samples, and *persistent* if inconsistency has lasted for k_{hard} or more consecutive windows.

5.3.4 Fault Classifier

The fault classifier maps the diagnosis output to a policy state:

- **No inconsistency, high confidence** $\rightarrow$ Normal Activation.
- **Inconsistency, transient** $\rightarrow$ Soft Safe State.
- **Inconsistency, persistent** $\rightarrow$ Hard Safe State.
- **Both sensors degraded or faulted** $\rightarrow$ Degraded Mode.

5.4 ESP32-S3 FreeRTOS Task Model

The subsystem maps onto four FreeRTOS tasks:

Table 4: FreeRTOS task allocation.

Task	Core	Priority	Function
radar_rx_task	Core 0	High	UART receive, parser, freshness stamp
camera_task	Core 1	High	Frame capture, format, quality check
diagnosis_task	Core 0	Medium	Consistency check, scoring, classification
activation_gk_task	Core 0	Medium	Policy enforcement, output gating

The **Task Watchdog Timer (TWDT)** monitors `radar_rx_task` and `camera_task` for stalls [8]. A stalled task that does not call `esp_task_wdt_reset()` within the watchdog period (configured at 5 s) triggers a watchdog event, logged as a timing fault and escalating the diagnosis state.

6 Fault Model and Diagnosis Logic

6.1 Fault Taxonomy

Following Avizienis et al. [1], faults in this subsystem are classified along four axes: source, nature, duration, and detectability.

6.1.1 By Source

Sensing faults. Originate in the physical transducer or its immediate interface: radar clutter, camera low-light, radar multipath reflection.

Communication faults. Originate in data transport: UART packet drops, PSRAM bus contention, camera DMA buffer overrun.

Timing faults. Originate in task scheduling or resource contention: stale frames, over-long inference windows delaying the diagnosis task, watchdog events.

Environmental faults. Originate outside the sensor: fan, moving curtain, reflective surface, sudden lighting change.

6.1.2 By Duration

Transient faults. Last one or a few decision windows; may self-resolve. Examples: brief radar glitch from a passing vehicle, single dropped UART packet.

Persistent faults. Persist across k_{hard} consecutive windows. Examples: camera initialization failure, misaligned radar installation.

6.1.3 SDC-Like (Silent Data Corruption) Faults

A particularly dangerous sub-class is outputs that appear valid but conflict with system state. Example: the radar timestamp is fresh and the presence bit is set, but the camera has been dark for 10 seconds, and room-level ambient sensors suggest no occupancy. These faults are addressed by invariant checks: if presence is asserted for longer than $T_{\text{max_present}}$ without any camera visual confirmation event, a fault is flagged.

6.2 Fault Detection Rules

Table 5: Fault detection rules and response actions.

Fault Class	Example	Observable Symptom	Response
Camera false negative	Low light or occlusion	Radar present; camera absent	Revalidate; soft safe state
Radar false positive	Fan/curtain/reflection	Radar present; camera no user	Delay activation; request confirmation
UART/transport fault	Dropped radar packet	Missing/stale radar timestamp	Parser reset; suppress activation
Timing fault	Camera task delay	Fusion uses stale frame	Reject frame; watchdog recovery
Persistent sensor fault	Camera init failure	Repeated visual failure	Hard safe state; isolate camera
SDC-like output	Stale plausible data	Valid-looking data conflicts with state	Invariant check; roll-back health state

6.3 Diagnosis Decision Flow

Figure 2 shows the logic applied each decision window.

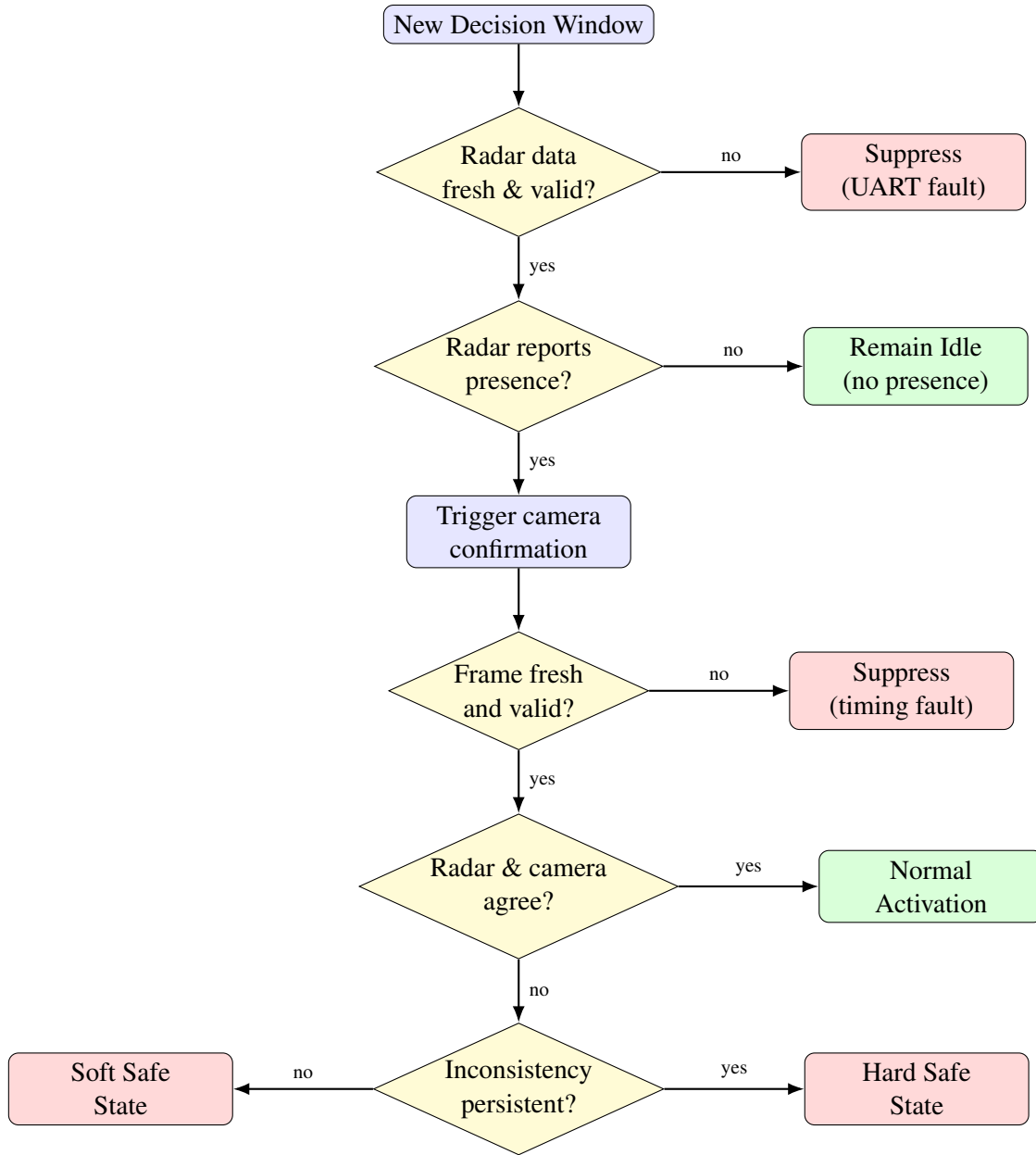


Figure 2: Diagnosis decision logic per activation window.

7 Safe-State and Recovery Policy

7.1 State Machine Overview

The activation subsystem is modeled as a state machine over seven states. Figure 3 illustrates the transitions.

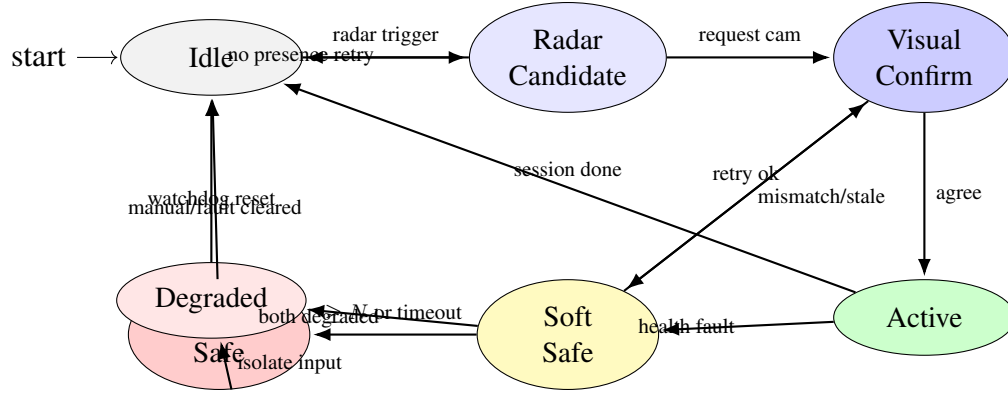


Figure 3: Activation state machine. Safe states (yellow/red) suppress activation.

7.2 Soft Safe State

The soft safe state is entered when a **transient inconsistency** is detected.

- Full activation (camera inference, audio, actuation) is **suppressed**.
- The system collects another radar sample and camera frame.
- Timestamps and health scores are re-evaluated.
- If consistency is restored within the bounded time window T_{retry} , the system transitions back to VisualConfirm.
- If inconsistency persists beyond T_{retry} or repeats N times, the system escalates to Hard Safe State.

7.3 Hard Safe State

The hard safe state is entered when a **persistent inconsistency** is detected.

- Automatic activation is **blocked completely**.
- The low-confidence or faulted sensor input is **isolated** from the fusion computation.
- The system optionally notifies the user (e.g., OLED message or audio tone) that reduced sensing is in effect.
- Minimal power operation is maintained (radar sentinel continues; camera is paused).
- Exit requires either manual reset or watchdog-supervised sensor re-initialization.

7.4 Recovery Block for Visual Confirmation

A classical **recovery block** [11] is applied to the visual confirmation step:

1. **Primary confirmer:** face/body detector (ESP-DL HumanFaceDetect INT8). Acceptance test: confidence score $> \tau_{\text{face}}$ and bounding box within valid range.
2. **Alternate confirmer:** frame-difference motion detector. If the primary confirmer fails or times out, fall back to whether significant pixel-delta motion is detected in the expected detection zone.
3. **Acceptance test for alternate:** motion delta $> \delta_{\text{min}}$ in the center region.
4. **Recovery failure:** if alternate also fails acceptance, the system remains in Soft Safe State and decrements the retry counter.

This matches the recovery-block design philosophy: *try the primary method; if its output passes the acceptance test, proceed; otherwise try the alternate; if the alternate also fails, enter safe state.*

7.5 Confidence Score Fusion

The per-sensor confidence score from Equation 1 is computed as follows for each decision window:

$$S_i = \begin{cases} 1.0 & \text{sensors agree on presence/absence} \\ 0.0 & \text{disagreement} \end{cases} \quad (2)$$

$$F_i = \max\left(0, 1 - \frac{t_{\text{now}} - t_{\text{sample}}}{T_{\text{stale}}}\right) \quad (3)$$

$$T_i = \begin{cases} 1.0 & \text{TWDT not triggered, no queue overflow} \\ 0.0 & \text{watchdog or overflow detected} \end{cases} \quad (4)$$

If $C_i < \tau_C$ (threshold, e.g., 0.6), the system suppresses activation regardless of the binary presence decision.

8 Evaluation Plan and Metrics

8.1 Evaluation Design

Four activation controllers are compared:

1. **Radar-only baseline.** Activate on any radar presence signal.
2. **Camera-only baseline.** Activate on any face-detected frame.

3. **Naive AND/OR fusion.** Activate on AND (strict) or OR (permissive) of both sensor outputs; no fault diagnosis.
4. **Proposed fault-tolerant fusion.** Confidence scoring, cross-sensor diagnosis, transient/persistent classification, soft/hard safe states.

8.2 Fault Injection Tests

Following Moradi and Denil [4], faults are injected at the software level by intercepting sensor data before it reaches the diagnosis layer:

Table 6: Fault injection test cases.

Fault Type	Injection Method	Target Metric
Radar packet drops	Drop 1-in- N UART frames randomly	FAR, MDR, fault coverage
Radar framing error	Corrupt header/footer bytes	Fault coverage, detection latency
Camera frame blackout	Replace frame with all-zero buffer	MDR, soft safe state rate
Camera motion blur	Convolve with Gaussian kernel	Fault coverage
Stale camera frame	Freeze timestamp for 500 ms	Timing fault detection rate
Induced task delay	Insert <code>vTaskDelay</code> in camera task	TWDT trigger rate
Low-light degradation	Scale frame luminance to 20%	Camera health score, MDR
UART transport silence	Block UART RX for 2 s	Hard safe state transition time

8.3 Environmental Stress Tests

Real-environment tests are conducted in the target room with the physical hardware:

- Empty room (no occupant): tests false activation rate.
- Single user entering and exiting the detection zone.
- Static seated user: tests stationary detection.
- Low-light condition (lights off): tests camera degradation handling.
- Reflective surface (mirror placed in radar field): tests radar clutter.
- Moving fan: tests nuisance motion rejection.

8.4 Metrics

$$\text{FAR} = \frac{\text{False Activations}}{\text{Total Non-user Trials}} \quad (5)$$

$$\text{MDR} = \frac{\text{Missed Activations}}{\text{Total Valid User Trials}} \quad (6)$$

$$\text{Fault Coverage} = \frac{\text{Detected Injected Faults}}{\text{Total Injected Faults}} \quad (7)$$

Additional metrics:

- **Detection latency:** time from fault injection to fault flag.
- **Time-to-safe-state:** time from fault detection to safe-state entry.
- **Recovery time:** time from fault clearance to Normal Activation.
- **Camera duty cycle:** fraction of time camera is actively capturing (proxy for energy and privacy exposure).
- **Activation latency overhead:** added delay of fault-tolerant fusion relative to the naive baseline.

8.5 Expected Results

The proposed system is expected to:

- Reduce FAR relative to radar-only (which has no semantic check).
- Reduce MDR relative to strict AND fusion (which fails if either sensor has a transient fault).
- Achieve higher fault coverage than naive fusion (which does not diagnose disagreements).
- Incur moderate activation latency overhead (<500 ms) due to revalidation.

The primary tradeoff is slightly higher **activation latency** because the soft-safe-state revalidation loop adds bounded delay before allowing activation. This tradeoff is acceptable for an assistive system where false activations carry privacy costs.

9 Analysis of Existing Solutions vs. Proposed Improvement

Table 7 compares all known approaches against the proposed system on six fault-tolerance dimensions.

Table 7: Comparison of known approaches and proposed improvement.

Dimension	Radar-only	Camera-only	Naive Fusion	Proposed
Heterogeneous redundancy	No	No	Yes (implicit)	Yes (explicit)
Cross-sensor diagnosis	No	No	No	Yes
Transient/persistent classification	No	No	No	Yes
Soft/hard safe states	No	No	No	Yes
Confidence/freshness scoring	No	No	No	Yes
Fault-injection evaluation	No	No	No	Yes
Timing fault detection (TWDT)	No	No	No	Yes
Recovery block for vision	No	No	No	Yes

The gaps in existing approaches translate directly to failure modes:

- **Radar-only gap:** no independent validation means radar clutter directly causes false activation with no recourse.
- **Camera-only gap:** high resource consumption and environmental sensitivity create excessive missed activations in low-light conditions.
- **Naive fusion gap:** AND logic causes MDR to increase whenever one sensor experiences a transient fault; OR logic causes FAR to increase when either sensor is noisy. Neither diagnoses the disagreement.

The proposed health-aware activation controller addresses all three gaps without requiring a GPU or deep-learning pipeline.

10 Expected Contribution and Limitations

10.1 Expected Contributions

1. **Reduced false activation rate** relative to radar-only activation, because the camera provides semantic rejection of clutter.
2. **Reduced missed activation rate** relative to strict naive AND fusion, because transient faults enter a soft safe state and retry rather than blocking immediately.
3. **Reduced privacy and energy exposure** relative to camera-heavy activation, because the camera is activated on-demand only after radar trigger, and suppressed during faults.
4. **Explicit fault coverage** measured by fault injection, providing a quantified dependability characterization rather than only a nominal accuracy number.
5. **Demonstration of classical fault-tolerance principles**—redundancy, error detection, diagnosis, recovery blocks, and graceful degradation—applied to a modern ESP32-S3 edge AI activation pipeline.

10.2 Why This Matters

For a privacy-preserving assistive platform, ambiguity should not automatically trigger camera, audio, or actuator activation. The system should explicitly represent uncertainty and enter a safe state until confidence improves. This is the correct fault-tolerant design stance for a system where false activation carries privacy costs and missed activation carries assistance costs.

10.3 Limitations

- The project uses a low-cost 24 GHz UART-output mmWave module, not a research-grade radar with raw radar cubes. Advanced radar OOD algorithms such as HOOD [5] require richer radar data (Doppler profiles, range-Doppler maps) and are used here as conceptual motivation rather than directly reproduced.
- The evaluation is room-scale and involves limited occupancy scenarios and limited environmental variation. Generalization to multi-room, multi-occupant, or outdoor settings is not evaluated.
- The activation latency overhead from soft-safe-state revalidation is bounded but nonzero. For applications requiring sub-100 ms activation latency, this tradeoff would need re-evaluation.
- The confidence weights w_s , w_f , w_t and thresholds τ_C , k_{hard} , T_{stale} are tuned empirically in the target environment. Automatic threshold adaptation is left for future work.

11 Conclusion

This report has presented a fault-tolerant activation subsystem for a room-scale assistive sensing platform built on the ESP32-S3. The system addresses a problem that is distinct from pure sensing: making the activation decision dependable under realistic embedded faults, not merely optimizing detection accuracy under nominal conditions.

The proposed architecture uses:

- **Heterogeneous redundancy** (radar + camera) to cover the failure modes of each individual modality.
- **Cross-sensor consistency checking** to detect disagreements without immediately committing to either sensor's output.
- **Temporal redundancy** (k-sample revalidation) to distinguish transient from persistent faults.
- **Confidence and freshness scoring** to weight sensor inputs by their current reliability.
- **Soft and hard safe states** to implement fail-safe behavior at two levels of fault severity.
- **Recovery blocks** for the visual confirmation step.
- **Watchdog-based timing fault detection** using the ESP-IDF TWDT.

- **Fault-injection evaluation** to characterize dependability beyond nominal accuracy.

These are classical fault-tolerance principles applied to a modern, constrained, embedded edge AI system. The contribution is not a new sensing algorithm, but a new architectural pattern for making sensing-based activation decisions trustworthy.

For privacy-preserving assistive systems, this trustworthiness is not optional—it is a core requirement. A system that can activate itself incorrectly, or that silently fails to activate, violates the user’s autonomy, privacy, and ability to rely on the assistant. The fault-tolerant activation boundary proposed here is a step toward embedded edge AI systems that are not only capable but also dependable.

References

- [1] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, “Basic Concepts and Taxonomy of Dependable and Secure Computing,” *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, Jan.–Mar. 2004.
- [2] S. Li and R. Hishiyama, “An Indoor People Counting and Tracking System Using mmWave Sensor and Sub-sensors,” *IFAC-PapersOnLine*, 2023, doi: 10.1016/j.ifacol.2023.10.577.
- [3] Y. Yu, H. R. Karimi, L. Gelman, J. Tian, and P. Mei, “A Novel Multi-Source Sensor Correlation Adaptive Fusion Framework with Uncertainty Quantification for Intelligent Fault Diagnosis,” *Reliability Engineering & System Safety*, vol. 267, 2026, Art. no. 111812, doi: 10.1016/j.ress.2025.111812.
- [4] M. Moradi and J. Denil, “Machine-Learning Assisted Model-Implemented Fault Injection,” *CEUR Workshop Proceedings*, 2020.
- [5] S. M. Kahya, M. S. Yavuz, and E. Steinbach, “HOOD: Real-Time Human Presence and Out-of-Distribution Detection Using FMCW Radar,” *IEEE Transactions on Radar Systems*, 2025, doi: 10.1109/TRS.2024.3514840.
- [6] N. Zhang, H. Li, A. Zahid, Y. Tian, and W. Li, “A Robust mmWave Radar Framework for Accurate People Counting and Motion Classification,” *Sensors*, vol. 26, no. 4, Art. no. 1289, 2026, doi: 10.3390/s26041289.
- [7] Espressif Systems, *ESP32-S3 Series Datasheet and Technical Reference Manual*, 2023. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [8] Espressif Systems, “Watchdogs,” *ESP-IDF Programming Guide for ESP32-S3*, 2024. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/wdts.html>
- [9] Espressif Systems, *esp32-camera: ESP32 Camera Driver*, GitHub repository. Available: <https://github.com/espressif/esp32-camera>

- [10] Seeed Studio, “24 GHz mmWave Sensor for XIAO: Human Static Presence,” 2023. Available: <https://wiki.seeedstudio.com/Seeed-Studio-MR24HPC1-mmWave-Human-Static-Presence-Module-Lite/>
- [11] B. Randell, “System Structure for Software Fault Tolerance,” *IEEE Transactions on Software Engineering*, vol. SE-1, no. 2, pp. 220–232, Jun. 1975.